

ЛР1_завдання_студенту

September 21, 2026

1 Лабораторна робота № 1

1.1 Вхідний аналіз набору даних

Дисципліна: Інтелектуальний аналіз та візуалізація даних (СК26) **Тривалість:** 2 год аудиторно + 3 год самостійної роботи • **Максимальна оцінка:** 4 бали

1.1.1 Мета роботи

Пригадати роботу в Jupyter Notebook, бібліотеці pandas та побудову графіків — і зробити це не на навчальному прикладі, а на реальному файлі, який ніхто не готував спеціально для вас.

Ця робота виконується **до першої лекції**. Нічого нового знати не потрібно: усе, що знадобиться, ви вже проходили в попередніх курсах. Усі поняття, яких ви тут торкнетесь мимохідь, отримають назви й пояснення на лекціях 3 і 4.

1.1.2 Дані

Файл 20171129_FINAL_FINAL.xlsx, аркуш Sheet1 — спостереження космічної погоди за 28 серпня — 22 вересня 2017 року, зібрані з трьох різних джерел.

Файл видається **як є**, рівно в тому вигляді, у якому його колись вивантажили. Ніхто не приводив його до ладу, і саме в цьому сенс: справжні дані здебільшого надходять саме такими.

1.1.3 Що ви маєте зробити

Етап	Зміст	Де
1	Розібратися, що взагалі лежить у файлі	аудиторно
2	Виділити три таблиці й привести їх до робочого вигляду	аудиторно

Етап	Зміст	Де
3	Скласти паспорт даних: що є, чого бракує, що виглядає дивно	аудиторно
4	Об'єднати три таблиці в одну	аудиторно
5	Побудувати п'ять графіків, кожен — відповідь на питання	аудиторно + самостійно
6	Порівняти два графіки одних і тих самих даних	самостійно
7	Записати три спостереження про дані	самостійно
8	Декларація про використання інструментів ШІ	самостійно

1.1.4 Що здається

1. Цей файл `.ipynb` із заповненими розв'язками. **Ноутбук має виконуватися згори донизу на чистому ядрі** (Kernel □ Restart & Run All) без помилок — це перевіряється першою дією на захисті.
2. Файл `master_hourly.csv` — об'єднана таблиця з етапу 4.
3. Експорт ноутбука у HTML або PDF.
4. Заповнена декларація (етап 8). Без неї фінальна самоперевірка не проходить.

1.1.5 Критерії оцінювання

Бали	Що досягнуто
0	Дані не опрацьовано, графіки не побудовано або побудовано з грубими помилками
1	Таблиці прочитано й кілька графіків побудовано, але типи графіків не відповідають даним, дивних значень не помічено
2	Таблиці розібрано й об'єднано, графіки коректні та підписані, але висновків під ними немає або вони формальні
4	Усе вище виконано, знайдено й пояснено дивні значення, під кожним графіком є змістовний висновок про дані

Головне про оцінку: бали ставляться не за наявність графіка, а за речення під ним. Код, який нічого не пояснює, коштує двох балів з чотирьох.

1.2 Нагадування про Jupyter

Якщо давно не працювали — три речі, які знадобляться сьогодні:

- Shift+Enter виконує клітинку, Esc A / Esc B додає клітинку вище / нижче, Esc M перетворює клітинку на текстову (Markdown), Esc Y — назад на код.
 - **Порядок виконання має значення.** Якщо ви правили клітинки в довільному порядку, результат може залежати від історії запусків. Перед здачею обов'язково **Kernel ▢ Restart & Run All** — так ви побачите ноутбук очима викладача.
 - Текстові клітинки — не прикраса. Висновки пишуться в них, а не в коментарях до коду.
-

1.3 Крок 0. Ваші дані та варіант

Заповніть три змінні й виконайте обидві клітинки. Друга надрукує **ваше індивідуальне завдання** — воно відрізняється від завдання сусіда.

```
[13]: ПІБ = 'Колбасюк Данило Іванович'      # напр. 'Шевченко Тарас'
      ГРУПА = 'ШІ-32'      # напр. 'КН-31'
      ВАРІАНТ = 6      # ваш номер за списком групи

      assert ПІБ and ГРУПА and ВАРІАНТ, 'Заповніть ПІБ, ГРУПУ та ВАРІАНТ'
```

```
[14]: # --- клітинку не редагувати: вона видає ваше персональне завдання▢
      ↵-----
      ПОТОКИ = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8',▢
      ↵'E>2.0']

      ПАРИ = [('швидкість вітру', 'густина вітру'),
              ('швидкість вітру', 'потік P>10'),
              ('густина вітру', 'потік P>1'),
              ('радіопотік F10.7', 'потік P>10'),
              ('радіопотік F10.7', 'швидкість вітру'),
              ('потік E>0.8', 'потік P>10')]

      АГРЕГАТИ = ['середнє', 'максимум', 'медіана', 'мінімум']

      ПИТАННЯ = [
          'Скільки годин за все вікно ваш потік перевищував своє власне▢
          ↵медіанне значення?',
          'У яку добу середнє значення вашого потоку було найбільшим? А▢
          ↵найменшим?',
          'О котрій годині доби потік E>0.8 у середньому найбільший?▢
          ↵Побудуйте графік середнього за годинами доби.',
```

```

        'Яка частка рядків об'єднаної таблиці має пропуск хоча б в одному
        ↳стовпці?',
        'У який день радіопотік F10.7 був найбільшим і чи збігається цей
        ↳день з піком вашого потоку?',
        'Порівняйте медіану вашого потоку за перший і за другий тиждень
        ↳спостережень. У скільки разів вони відрізняються?',
        'Скільки годин поспіль тривала найдовша неперервна серія
        ↳«активних» годин (група з етапу 4)?',
        'Який зі стовпців об'єднаної таблиці має найбільший розкид
        ↳відносно власного середнього?',
        'Яку частку сумарного значення вашого потоку за все вікно дали 24
        ↳години навколо піку?',
        'Скільки різних діб потрапило у групу «активні» і скільки – у
        ↳групу «дуже активні»? ',
    ]

i = int(ВАРІАНТ)
завдання = {
    'ваш потік': ПОТОКИ[(i * 3 + 1) % 8],
    'ваша пара для графіка 3': ' та '.join(ПАРИ[(i * 5 + 2) %
    ↳6]),
    'ваш агрегат для етапу 4': АГРЕГАТИ[(i * 5 + 1) % 4],
    'ваше додаткове питання': ПИТАННЯ[(i * 11 + 5) % 10],
}

print('ІНДИВІДУАЛЬНЕ ЗАВДАННЯ · %s, %s, варіант %d' % (ПІБ, ГРУПА, i))
print('=' * 78)
for k, v in завдання.items():
    print('%-28s %s' % (k + ': ', v))

```

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ · Колбасюк Данило Іванович, ШІ-32, варіант 6

```

ваш потік: P>30
ваша пара для графіка 3: густина вітру та потік P>1
ваш агрегат для етапу 4: мінімум
ваше додаткове питання: У яку добу середнє значення вашого потоку
↳було
найбільшим? А найменшим?

```

Далі за текстом **«ваш потік»** означає стовпець, названий у першому рядку завдання.

```

[15]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option('display.width', 200)
pd.set_option('display.max_columns', 40)

```

```
plt.rcParams.update({'figure.dpi': 110, 'axes.spines.top': False,
                    'axes.spines.right': False, 'font.size': 9})

ФАЙЛ = '20171129_FINAL_FINAL.xlsx'
```

2 Етап 1. Що взагалі лежить у файлі

Перше правило роботи з незнайомими даними: спершу подивитися, потім рахувати.

2.0.1 Завдання 1.1

Спробуйте прочитати аркуш Sheet1 звичайним способом — `pd.read_excel(ФАЙЛ)` — і виведіть перші рядки та типи стовпців.

Очікуваний результат: результат буде дивним. Подивіться на назви стовпців і на те, які там типи даних. Поясніть у наступній клітинці, чому так вийшло.

Відповідь 1.1: (чому звичайне читання дало саме такий результат) При читанні ми не обробили самі дані, тому вивелась повна таблиця, неназвані стовпчики відповідно Unnamed: #, а пусті клітинки - NaN.

2.0.2 Завдання 1.2

Прочитайте аркуш ще раз, але **без заголовка**: `pd.read_excel(ФАЙЛ, header=None)`. Виведіть його розмір і подивіться на перші 30 рядків.

```
[16]: # ВАШ КОД
RAW = pd.read_excel(ФАЙЛ, header=None)
print(RAW.shape)
print(RAW.head(30))
```

```
(7226, 32)
0 1 2 3 4
5 6 7 8 9 10 11 12 13 14
15 \
0 NaN ftp://ftp.swpc.noaa.gov/pub/lists/particle/ NaN NaN
NaN NaN NaN NaN NaN NaN NaN NaN NaN
NaN NaN
1 NaN NaN NaN NaN
NaN NaN
NaN NaN NaN NaN NaN NaN NaN NaN
NaN NaN
2 NaN :Data_list: 20170901_Gs_part_5m.txt NaN NaN
NaN
```

NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
3	NaN					:Created: 2017 Sep 02 0011 UTC	NaN	NaN	□
↪NaN									
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
4	NaN	# Prepared by the U.S. Dept. of Commerce, NOAA...	NaN	NaN	NaN		NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
5	NaN	# Please send comments and suggestions to SWPC...	NaN	NaN	NaN		NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
6	NaN						#	NaN	NaN
↪NaN									□
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
7	NaN	# Label: P > 1 = Particles at >1 Mev	NaN	NaN			NaN	NaN	□
↪NaN									
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
8	NaN	# Label: P > 5 = Particles at >5 Mev	NaN	NaN			NaN	NaN	□
↪NaN									
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
9	NaN	# Label: P >10 = Particles at >10 Mev	NaN	NaN			NaN	NaN	□
↪NaN									
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
10	NaN	# Label: P >30 = Particles at >30 Mev	NaN	NaN			NaN	NaN	□
↪NaN									
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
11	NaN	# Label: P >50 = Particles at >50 Mev	NaN	NaN			NaN	NaN	□
↪NaN									
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
12	NaN	# Label: P>100 = Particles at >100 Mev	NaN	NaN			NaN	NaN	□
↪NaN									
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
13	NaN	# Label: E>0.8 = Electrons at >0.8 Mev	NaN	NaN			NaN	NaN	□
↪NaN									
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	□
↪NaN	NaN								
14	NaN	# Label: E>2.0 = Electrons at >2.0 Mev	NaN	NaN			NaN	NaN	□
↪NaN									

NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
15	NaN			# Label: E>4.0 = Electrons at >4.0 Mev			NaN	NaN	NaN	NaN
↪NaN										
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
16	NaN			# Units: Particles = Protons/cm2-s-sr			NaN	NaN	NaN	NaN
↪NaN										
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
17	NaN			# Units: Electrons = Electrons/cm2-s-sr			NaN	NaN	NaN	NaN
↪NaN										
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
18	NaN					# Source: GOES-15	NaN	NaN	NaN	NaN
↪NaN										
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
19	NaN					# Location: W135	NaN	NaN	NaN	NaN
↪NaN										
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
20	NaN			# Missing data: -1.00e+05			NaN	NaN	NaN	NaN
↪NaN										
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
21	NaN						NaN	NaN	NaN	NaN
↪NaN										
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
22	NaN	5-minute	GOES-15	Solar Particle and Electron ...			NaN	NaN	NaN	NaN
↪NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
23	NaN						NaN	NaN	NaN	NaN
↪NaN										
Modified	Seconds		NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪	NaN	NaN								
24	NaN						# UT	C	D	ate
↪Time										
Julian	of the		NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
↪NaN	NaN									
25	NaN						# YR	MO	DA	
↪HHMM										
Day	Day	P > 1	P > 5	P >10	P >30	P >50	P>100	E>0.8	E>2.	
↪0	NaN									

26	NaN								2017	8	28	☐
	↪ 0											
57993		0	12	0.26	0.18	0.114	0.104	0.0793	23600			☐
	↪ 1180	NaN										
27	NaN								2017	8	28	☐
	↪ 5											
57993		300	19.4	0.69	0.314	0.112	0.0612	0.0251	23100			☐
	↪ 1160	NaN										
28	NaN								2017	8	28	☐
	↪ 10											
57993		600	8.48	0.168	0.13	0.0717	0.0614	0.0304	22200			☐
	↪ 1090	NaN										
29	NaN								2017	8	28	☐
	↪ 15											
57993		900	15.4	0.566	0.198	0.0926	0.0568	0.0251	20400			☐
	↪ 997	NaN										
									16	17	18	
19	20	21	22	23	24	25						☐
	↪	26										
27	28	29		30		31						
0								NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN		http://umtof.umd.				
	↪ edu/pm/crn/											
NaN	NaN	NaN		NaN		NaN						
1								NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN						☐
	↪	NaN										
NaN	NaN	NaN		NaN		NaN						
2								NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN		SOHO CELIAS☐				
	↪ Proton Monitor											
NaN	NaN	NaN		NaN		NaN						
3								NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN		Solar Wind Parameters for Carrington☐				
	↪ Rotation ...											
NaN	NaN	NaN		NaN		NaN						
4								NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN		Proton☐				
	↪ speed [km/sec]											
NaN	NaN	NaN		NaN		NaN						
5								NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN		Proton density [protons per cubic☐				
	↪ centimeter]											
NaN	NaN	NaN		NaN		NaN						
6								NaN	NaN	NaN		

[illegible]

18	#	Prepared by the U.S. Dept. of Commerce, NOA...	NaN	NaN				
NaN	NaN	NaN	NaN	NaN	NaN	NaN		
		NaN						
NaN	NaN	NaN	NaN	NaN	NaN			
19	#	Please send comments and suggestions to SWP...	NaN	NaN				
NaN	NaN	NaN	NaN	NaN	NaN	NaN		
		NaN						
NaN	NaN	NaN	NaN	NaN	NaN			
20						NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN		
		NaN						
NaN	NaN	NaN	NaN	NaN	NaN			
21						NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN		
		NaN						
NaN	NaN	NaN	NaN	NaN	NaN			
22						NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN		
		NaN						
NaN	NaN	NaN	NaN	NaN	NaN			
23						Quarterly Daily Solar Data	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN		
		NaN						
NaN	NaN	NaN	NaN	NaN	NaN			
24						NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN		
		NaN						
NaN	NaN	NaN	NaN	NaN	NaN			
25						NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN		
		NaN						
NaN	NaN	NaN	NaN	NaN	NaN			
26						NaN	Month	Date
		Radio Flux						
10.7	NaN	NaN	NaN	NaN	NaN	NaN		
		#						
YR	MO	DA	HH	SPEED	DENSITY			
27						2017	8	28
82	NaN	NaN	NaN	NaN	0.0	0.0		
		17						
8	28	0	285		3			
28						2017	8	29
84	NaN	NaN	NaN	NaN	1.0	1.0		
		17						
8	28	1	305		3.4			
29						2017	8	30

```
87 NaN NaN NaN NaN 2.0 2.0
    ↪      17
8    28    2    309    1.7
```



2.0.3 Завдання 1.3

Для кожного стовпця порахуйте, скільки в ньому заповнених клітинок (`RAW.notna().sum()`), і виведіть результат.

*Очікуваний результат: 32 числа. Серед них будуть нулі та згустки великих значень. За цими згустками видно, що в аркуші **не одна таблиця, а кілька**, вставлені поруч.*

Питання: скільки таблиць в аркуші і які стовпці займає кожна?

```
[17]: # ВАШ КОД
      print(RAW.notna().sum())
```

```
0      0
1    7223
2    7202
3    7202
4    7202
5    7203
6    7203
7    7198
8    7198
9    7198
10   7198
11   7198
12   7198
13   7198
14   7198
15      0
16     32
17     26
18     26
19     26
20      0
21      0
22      0
23      0
24    601
25    601
26    608
27    602
28    602
29    602
30    602
31    602
```

dtype: int64

Відповідь 1.3: Глянемо на 'пропуски' в згустках (нулі), по них бачимо, що є 3 таблиці, в першій 13 стовпчиків (2-14), в другій 4 стовпчика (17-20), в третій 8 стовпчиків (25-32)

2.0.4 Завдання 1.4

Над кожною таблицею є кілька текстових рядків — це службовий опис від тих, хто дані збирав. Виведіть їх і випишіть для кожної таблиці **джерело даних та одиниці вимірювання**.

Це найкорисніші п'ять хвилин роботи: жодної з цих відомостей у самій таблиці немає, і без них ви не знатимете, у чому вимірюєте.

```
[18]: print(RAW.iloc[0:20,1]) # Таблиця 1
      print(RAW.iloc[13:20,16]) # Таблиця 2
      print(RAW.iloc[0:7,26]) # Таблиця 3
```

```
0      ftp://ftp.swpc.noaa.gov/pub/lists/particle/
1                                          NaN
2      :Data_list: 20170901_Gs_part_5m.txt
3      :Created: 2017 Sep 02 0011 UTC
4      # Prepared by the U.S. Dept. of Commerce, NOAA...
5      # Please send comments and suggestions to SWPC...
6                                          #
7      # Label: P > 1 = Particles at >1 Mev
8      # Label: P > 5 = Particles at >5 Mev
9      # Label: P >10 = Particles at >10 Mev
10     # Label: P >30 = Particles at >30 Mev
11     # Label: P >50 = Particles at >50 Mev
12     # Label: P>100 = Particles at >100 Mev
13     # Label: E>0.8 = Electrons at >0.8 Mev
14     # Label: E>2.0 = Electrons at >2.0 Mev
15     # Label: E>4.0 = Electrons at >4.0 Mev
16     # Units: Particles = Protons/cm2-s-sr
17     # Units: Electrons = Electrons/cm2-s-sr
18     # Source: GOES-15
19     # Location: W135
Name: 1, dtype: object
13     ftp://ftp.swpc.noaa.gov/pub/indices/old_indice...
14                                          NaN
15     Product: Daily Solar Data      quar_DSD.txt
16     :Issued: 0825 UT 04 Oct 2017
17     #
18     # Prepared by the U.S. Dept. of Commerce, NOAA...
19     # Please send comments and suggestions to SWP...
Name: 16, dtype: object
0      http://umtof.umd.edu/pm/crn/
1                                          NaN
```

```

2                                SOHO CELIAS Proton Monitor
3    Solar Wind Parameters for Carrington Rotation ...
4                                Proton speed [km/sec]
5    Proton density [protons per cubic centimeter]
6                                NaN
Name: 26, dtype: object

```

Відповідь 1.4:

Таблиця	Джерело	Одиниці вимірювання
A	ftp://ftp.swpc.noaa.gov/pub/lists/data/19970101	Протони/cm ² *s*sr, Електрони = Електрони/cm ² *s*sr
B	ftp://ftp.swpc.noaa.gov/pub/indices/indices/20170101_DSD.txt	Рівень сонячної активності за годину
C	http://umtof.umd.edu/pm/crn/	км/с, протони на см ²

3 Етап 2. Три таблиці в робочому вигляді

У файлі три таблиці. Ось що в них лежить — решту ви вже знайшли самі:

Таблиця	Стовпці аркуша	Що це	Один рядок =
A	B–O	вимірювання із супутника GOES-15: потоки частинок різної енергії	5 хвилин
B	Q–T	добовий показник сонячної активності F10.7	доба
C	Y–AF	швидкість і густина сонячного вітру	година

3.0.1 Завдання 2.1

Виділіть кожну таблицю в окремий DataFrame — назвіть їх A, B і C. Дайте стовпцям осмислені назви й перетворіть значення на числа (`pd.to_numeric(..., errors='coerce')` стане в пригоді).

```

[19]: A = RAW.iloc[26:7226, 1:15]
      B = RAW.iloc[27:52, 16:20]
      C = RAW.iloc[27:628, 24:32]

```

```

A.columns = ['year', 'month', 'day', 'hhmm', 'Modified Julian Day',
↳ 'Seconds of the day', 'P>1', 'P>5', 'P>10', 'P>30', 'P>50',
↳ 'P>100', 'E>0.8', 'E>2.0']
B.columns = ['year', 'month', 'day', 'Radio Flux 10.7']
C.columns = ['hour1', 'hour2', 'year', 'month', 'day', 'hour',
↳ 'Speed', 'Density']
C=C.drop(columns=['hour1', 'hour2'])
A=A.apply(pd.to_numeric, errors='coerce')
B=B.apply(pd.to_numeric, errors='coerce')
C=C.apply(pd.to_numeric, errors='coerce')

```

3.0.2 Завдання 2.2

У кожній таблиці дата й час записані **по-різному** й розкидані по кількох стовпцях. Зберіть для кожної таблиці один стовпець ts типу datetime.

Після цього обов'язково виведіть для кожної таблиці найранішу й найпізнішу дату.

Очікуваний результат: усі три таблиці мають лежати в межах серпня-вересня 2017 року. Якщо у вас вийшов інший рік — придивіться, як записано рік у тій таблиці, і виправте.

```

[20]: A['ts'] = pd.to_datetime(
    A['year'].astype(str) + "-" +
    A['month'].astype(str) + "-" +
    A['day'].astype(str) + " " +
    A['hhmm'].astype(str).str.zfill(4),
    format="%Y-%m-%d %H%M",
    errors="coerce"
)
A=A.drop(columns=['year', 'month', 'day', 'hhmm', 'Modified Julian
↳ Day', 'Seconds of the day'])
B['ts']=pd.to_datetime(B[['year', 'month', 'day']])
B=B.drop(columns=['year', 'month', 'day'])
C['year']=C['year']+2000
C['ts']=pd.to_datetime(C[['year', 'month', 'day', 'hour']])
C=C.drop(columns=['year', 'month', 'day', 'hour'])
print(A['ts'].min())
print(A['ts'].max())
print(B['ts'].min())
print(B['ts'].max())
print(C['ts'].min())
print(C['ts'].max())

```

```

2017-08-28 00:00:00
2017-09-21 23:55:00
2017-08-28 00:00:00
2017-09-21 00:00:00

```

2017-08-28 00:00:00

2017-09-22 00:00:00

Самоперевірка етапу 2. Клітинка має виконатися без помилок.

```
[21]: assert len(A) == 7200, 'у таблиці A має бути 7200 рядків, а є %d' % len(A)
      assert len(B) == 25, 'у таблиці B має бути 25 рядків, а є %d' % len(B)
      assert len(C) == 601, 'у таблиці C має бути 601 рядок, а є %d' % len(C)
      for назва, т in [('A', A), ('B', B), ('C', C)]:
          рік = pd.to_datetime(т['ts']).dt.year
          assert рік.min() == 2017 and рік.max() == 2017, 'у таблиці %s рік
          не 2017 – перевірте збирання дати' % назва
      print('Етап 2 пройдено')
```

Етап 2 пройдено

4 Етап 3. Паспорт даних

4.0.1 Завдання 3.1

Для кожної таблиці виведіть: розмір, типи стовпців, кількість пропусків у кожному стовпці, кількість повних дублікатів рядків і описові статистики (describe()).

```
[22]: print(A.shape)
      print(A.dtypes)
      print(A.isna().sum())
      print(A.duplicated().sum())
      print(A.describe())
      print(B.shape)
      print(B.dtypes)
      print(B.isna().sum())
      print(B.duplicated().sum())
      print(B.describe())
      print(C.shape)
      print(C.dtypes)
      print(C.isna().sum())
      print(C.duplicated().sum())
      print(C.describe())
```

(7200, 9)

P>1 float64

P>5 float64

P>10 float64

P>30 float64

```

P>50          float64
P>100         float64
E>0.8         float64
E>2.0         float64
ts            datetime64[us]
dtype: object
P>1           3
P>5           3
P>10          3
P>30          3
P>50          3
P>100         3
E>0.8         3
E>2.0         3
ts            0
dtype: int64
0

          P>1          P>5          P>10          P>30          P>50
P>100      E>0.8      E>2.0      ts
count  7197.000000  7197.000000  7197.000000  7197.000000  7197.000000
7197.000000  7197.000000  7197.000000  7200
mean    689.325064  156.626057  76.474020  18.838604  7.446840
1.65929  81862.741801  11401.502123  2017-09-09 11:57:30
min      0.429000  0.151000  0.117000  0.057600  0.045500
0.02240  4.300000  0.133000  2017-08-28 00:00:00
25%      11.700000  0.279000  0.186000  0.098400  0.059700
0.02630  22200.000000  552.000000  2017-09-03 05:58:45
50%      55.400000  0.632000  0.376000  0.146000  0.086600
0.03630  68600.000000  4290.000000  2017-09-09 11:57:30
75%      560.000000  107.000000  35.800000  0.990000  0.249000
0.06700  143000.000000  14100.000000  2017-09-15 17:56:15
max     32500.000000  2890.000000  1330.000000  399.000000  191.000000
61.50000  259000.000000  110000.000000  2017-09-21 23:55:00
std     1625.968447  390.921861  222.362627  67.265595  28.620524
7.15889  65728.005824  16920.617446  NaN
(25, 2)
Radio Flux 10.7          int64
ts            datetime64[us]
dtype: object
Radio Flux 10.7      0
ts                  0
dtype: int64
0

          Radio Flux 10.7          ts
count          25.000000          25
mean          92.680000  2017-09-09 00:00:00
min           71.000000  2017-08-28 00:00:00
25%           74.000000  2017-09-03 00:00:00

```



```

50%          84.000000  2017-09-09 00:00:00
75%          107.000000 2017-09-15 00:00:00
max           140.000000 2017-09-21 00:00:00
std           22.206831                NaN
(601, 3)
Speed                int64
Density              float64
ts                  datetime64[us]
dtype: object
Speed              0
Density            0
ts                 0
dtype: int64
0

```

	Speed	Density	ts
count	601.000000	601.000000	601
mean	506.547421	2.907654	2017-09-09 12:00:00
min	-1.000000	-1.000000	2017-08-28 00:00:00
25%	447.000000	1.800000	2017-09-03 06:00:00
50%	517.000000	2.400000	2017-09-09 12:00:00
75%	584.000000	3.400000	2017-09-15 18:00:00
max	806.000000	31.700000	2017-09-22 00:00:00
std	125.073291	2.637873	NaN

4.0.2 Завдання 3.2 — дивні значення

Подивіться на рядок min у таблиці describe() для таблиці C.

Швидкість і густина — фізичні величини, які **не можуть бути від'ємними**. Знайдіть від'ємні значення, порахуйте, скільки їх, і поверніться до службового опису з завдання 1.4: там написано, що вони означають.

Замініть ці значення на NaN і покажіть, як після цього змінилися середнє й мінімум.

Очікуваний результат: два числа «до» і «після» для кожного зі стовпців.

```

[23]: print("Before replacement")
      print((C['Speed']==-1).sum())
      print((C['Density']==-1).sum())
      print(f"Speed mean: {C['Speed'].mean()}")
      print(f"Speed minimum: {C['Speed'].min()}")
      print(f"Density mean: {C['Density'].mean()}")
      print(f"Density minimum: {C['Density'].min()}")
      C['Density']=C['Density'].replace(-1,np.nan)
      C['Speed']=C['Speed'].replace(-1,np.nan)
      print("After replacement")
      print((C['Speed']==-1).sum())
      print((C['Density']==-1).sum())
      print(f"Speed mean: {C['Speed'].mean()}")

```

```
print(f"Speed minimum: {C['Speed'].min()}")
print(f"Density mean: {C['Density'].mean()}")
print(f"Density minimum: {C['Density'].min()}")
```

Before replacement

8

8

Speed mean: 506.5474209650582

Speed minimum: -1

Density mean: 2.9076539101497505

Density minimum: -1.0

After replacement

0

0

Speed mean: 513.3946037099494

Speed minimum: 277.0

Density mean: 2.960370994940978

Density minimum: 0.1

Відповідь 3.2: -1 означає втрачені дані, таких було 8 рядків і вони робили середнє меншим, ніж воно є насправді.

4.0.3 Завдання 3.3 — питання на подумати

У таблиці A є стовпці, для яких `describe()` слухняно рахує середнє й стандартне відхилення, хоча ці числа не мають жодного сенсу.

Знайдіть їх і поясніть своїми словами, чому середнє для них безглузде.

Підказка: не кожне число в таблиці є результатом вимірювання.

Це питання не має однієї «правильної» відповіді з підручника — на нього ви відповідаєте з власного здорового глузду. Формальне пояснення буде на лекції 3.

[24]: A.describe()

```
[24]:
```

	P>1	P>5	P>10	P>30	
↪P>50					
P>100	E>0.8	E>2.0		ts	
count	7197.000000	7197.000000	7197.000000	7197.000000	7197.000000
↪000000					
7197.000000	7197.000000	7197.000000		7200	
mean	689.325064	156.626057	76.474020	18.838604	7.446840
↪446840					
1.65929	81862.741801	11401.502123	2017-09-09	11:57:30	
min	0.429000	0.151000	0.117000	0.057600	0.045500
↪045500					
0.02240	4.300000	0.133000	2017-08-28	00:00:00	

25%	11.700000	0.279000	0.186000	0.098400	0.
↪059700					
0.02630	22200.000000	552.000000	2017-09-03	05:58:45	
50%	55.400000	0.632000	0.376000	0.146000	0.
↪086600					
0.03630	68600.000000	4290.000000	2017-09-09	11:57:30	
75%	560.000000	107.000000	35.800000	0.990000	0.
↪249000					
0.06700	143000.000000	14100.000000	2017-09-15	17:56:15	
max	32500.000000	2890.000000	1330.000000	399.000000	191.
↪000000					
61.50000	259000.000000	110000.000000	2017-09-21	23:55:00	
std	1625.968447	390.921861	222.362627	67.265595	28.
↪620524					
7.15889	65728.005824	16920.617446			NaN

Відповідь 3.3: Це стосується стовпців часу. Середнє/стандартне відхилення якщо і обчислюється, то потім не зрозуміло, як його використати.

5 Етап 4. Об'єднання трьох таблиць в одну

Таблиці мають різний крок у часі: 5 хвилин, година і доба. Щоб працювати з ними разом, потрібна спільна сітка. Беремо **годину**.

Кожна таблиця потребує своєї дії:

1. **A: 5 хвилин** → **година**. Кілька рядків згортаються в один — це `resample` + агрегація.
2. **C: уже година**. Просте з'єднання по спільному стовпцю `ts` — `merge`.
3. **B: доба** → **година**. Навпаки, одне значення розтягується на 24 години доби.

5.0.1 Завдання 4.1

Зведіть таблицю A до погодинної сітки. Для кожного потоку порахуйте **два** числа за годину: середнє і максимум.

```
[25]: flux_cols = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100',
                  'E>0.8', 'E>2.0']

A_hourly = A.set_index('ts')[flux_cols].resample('1h').agg(['mean',
                  ↪ 'max'])
A_hourly.columns = [f'{col}_{stat}' for col, stat in A_hourly.columns]
A_hourly = A_hourly.reset_index()
A_hourly
```

[25]:

```
ts    P>1_mean  P>1_max  P>5_mean  P>5_max  \
↪P>10_mean
P>10_max  P>30_mean  P>30_max  P>50_mean  P>50_max  P>100_mean  \
↪P>100_max
E>0.8_mean  E>0.8_max  E>2.0_mean  \
0    2017-08-28 00:00:00    9.778333    19.40    0.324917    0.690    0.
↪192917
0.314    0.098850    0.2240    0.077583    0.1780    0.040692    0.0793
20941.666667    23600.0    1011.583333
1    2017-08-28 01:00:00    7.994167    13.60    0.291667    0.512    0.
↪156583
0.259    0.072583    0.0919    0.058700    0.0798    0.029950    0.0471
18091.666667    20600.0    713.000000
2    2017-08-28 02:00:00    6.099167    10.80    0.283000    0.479    0.
↪177667
0.321    0.084450    0.1210    0.069642    0.1110    0.035025    0.0595
14108.333333    15000.0    380.833333
3    2017-08-28 03:00:00    4.752500    7.76    0.236833    0.329    0.
↪166333
0.244    0.087658    0.1800    0.068092    0.1390    0.035517    0.0818
12766.666667    13100.0    267.916667
4    2017-08-28 04:00:00    4.158333    6.62    0.334917    0.539    0.
↪208333
0.309    0.084800    0.1530    0.072850    0.1420    0.039708    0.0694
11333.333333    12300.0    203.000000
..
...
...
595 2017-09-21 19:00:00    63.166667    93.10    0.372500    0.795    0.
↪229833
0.472    0.110642    0.1570    0.074300    0.1050    0.040192    0.0687
180250.000000    186000.0    65750.000000
596 2017-09-21 20:00:00    42.102500    78.90    0.397500    0.833    0.
↪226833
0.429    0.089325    0.1380    0.069983    0.1050    0.039692    0.0737
178750.000000    186000.0    63175.000000
597 2017-09-21 21:00:00    50.783333    71.70    0.489250    0.788    0.
↪317583
0.501    0.121358    0.2040    0.077467    0.1470    0.033417    0.0524
175833.333333    177000.0    58350.000000
598 2017-09-21 22:00:00    39.000000    48.80    0.350000    0.536    0.
↪200250
0.310    0.094358    0.1430    0.070633    0.1080    0.039600    0.0798
167750.000000    176000.0    49366.666667
599 2017-09-21 23:00:00    28.825000    42.10    0.465000    0.861    0.
↪257833
```

```
0.506    0.113575    0.1680    0.078108    0.1110    0.045125    0.0733
150083.333333    160000.0    37525.000000
```

```
      E>2.0_max
0      1180.0
1      907.0
2      475.0
3      284.0
4      234.0
..      ...
595    66800.0
596    70500.0
597    60300.0
598    54400.0
599    44400.0
```

```
[600 rows x 17 columns]
```

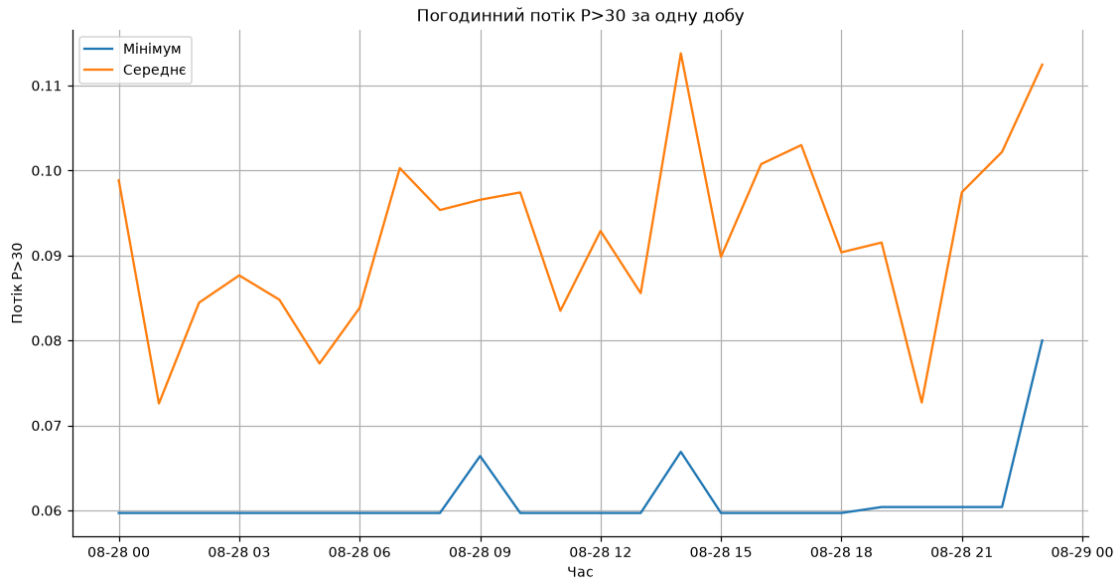
5.0.2 Завдання 4.2

У вашому завданні названо агрегат (середнє, максимум, медіана або мінімум).

Візьміть ваш потік за одну добу, порахуйте для нього по годинах ваш агрегат і ще один на ваш вибір, і побудуйте обидві криві на одному графіку.

Питання: чим вони відрізняються і в якій ситуації різниця між ними стане суттєвою?

```
[26]: day = A[A['ts'].dt.date == A['ts'].dt.date.iloc[0]].copy()
hourly = (day.set_index('ts')['P>30'].resample('1h').agg(['min',
↪ 'mean']))
plt.figure(figsize=(12, 6))
plt.plot(hourly.index, hourly['min'], label='Мінімум')
plt.plot(hourly.index, hourly['mean'], label='Середнє')
plt.xlabel('Час')
plt.ylabel('Потік P>30')
plt.title('Погодинний потік P>30 за одну добу')
plt.legend()
plt.grid(True)
plt.show()
```



Відповідь 4.2: В цьому випадку, мінімум значно нижчий за середнє. Це показує, що потік був досить нестабільним. Це важливо, коли нам потрібно бачити якісь коливання в значеннях, а маючи тільки середнє чи мінімум (або іншу комбінацію агрегатів), ми б ці дані не отримали.

5.0.3 Завдання 4.3

Об'єднайте три таблиці в одну — назвіть її M. Виведіть її розмір і всі рядки, у яких є хоча б один пропуск.

Зверніть увагу: таблиці закінчуються не в один і той самий момент. Тому результат залежить від того, який тип об'єднання ви оберете (`how='inner'` чи `how='outer'`). Спробуйте обидва, подивіться, чим відрізняється кількість рядків, і поясніть свій вибір.

```
[27]: B_hourly = (B.set_index('ts').resample('1h').ffill().reset_index())
M=(A_hourly.merge(C, on='ts', how='outer').merge(B_hourly, on='ts',
↪how='outer').sort_values('ts'))

print("Розмір M:", M.shape)
print("\nРядки з пропусками:")
print(M[M.isna().any(axis=1)])
```

Розмір M: (601, 20)

Рядки з пропусками:

```
ts    P>1_mean  P>1_max  P>5_mean  P>5_max  ↪
↪P>10_mean
P>10_max  P>30_mean  P>30_max  P>50_mean  P>50_max  P>100_mean  ↪
↪P>100_max
```

	E>0.8_mean	E>0.8_max	E>2.0_mean	\				
175	2017-09-04 07:00:00	6.465833	10.10	0.262083	0.593	0.		
↪171500								
0.294	0.085483	0.145	0.068558	0.1340	0.031717	0.0523		
90600.000000	100000.0	7410.833333						
176	2017-09-04 08:00:00	4.010833	5.08	0.256000	0.432	0.		
↪195833								
0.336	0.086625	0.145	0.070033	0.1340	0.036492	0.0546		
73483.333333	79300.0	5048.333333						
177	2017-09-04 09:00:00	3.268333	4.42	0.303417	0.518	0.		
↪220333								
0.302	0.105925	0.162	0.081975	0.1340	0.046483	0.0765		
60475.000000	68500.0	3715.833333						
178	2017-09-04 10:00:00	3.179167	4.74	0.252167	0.372	0.		
↪181833								
0.280	0.101900	0.150	0.081675	0.1200	0.054125	0.0894		
64383.333333	68400.0	4122.500000						
179	2017-09-04 11:00:00	5.274167	7.98	0.295083	0.657	0.		
↪186917								
0.276	0.089525	0.166	0.063983	0.1100	0.033633	0.0684		
79933.333333	88200.0	6345.833333						
180	2017-09-04 12:00:00	2.965000	4.85	0.217750	0.428	0.		
↪153500								
0.239	0.089217	0.179	0.071217	0.1440	0.037233	0.0679		
80516.666667	88700.0	6080.000000						
181	2017-09-04 13:00:00	3.061667	3.86	0.275833	0.551	0.		
↪192833								
0.329	0.104542	0.170	0.083783	0.1590	0.041342	0.0611		
76733.333333	85900.0	6015.833333						
182	2017-09-04 14:00:00	7.279167	10.80	0.231333	0.409	0.		
↪170583								
0.313	0.086508	0.178	0.070667	0.1660	0.037375	0.0888		
83025.000000	90200.0	8552.500000						
577	2017-09-21 01:00:00	37.408333	56.30	0.268250	0.375	0.		
↪196667								
0.335	0.116075	0.255	0.078192	0.1600	0.039483	0.0737		
159833.333333	164000.0	38200.000000						
578	2017-09-21 02:00:00	36.841667	48.60	0.357250	0.774	0.		
↪229333								
0.493	0.100842	0.154	0.071567	0.1290	0.038883	0.0737		
150583.333333	158000.0	32333.333333						
579	2017-09-21 03:00:00	27.325000	39.80	0.380333	0.678	0.		
↪198750								
0.411	0.093750	0.126	0.068617	0.0941	0.035458	0.0691		
135083.333333	143000.0	24566.666667						
580	2017-09-21 04:00:00	16.550000	29.80	0.393250	0.591	0.		
↪196417								

0.272	0.105617	0.185	0.071892	0.1110	0.031250	0.0420
120833.333333	131000.0	18641.666667				
581	2017-09-21 05:00:00	9.072500	15.20	0.533833	0.878	0.
→255917						
0.422	0.103892	0.148	0.070758	0.0987	0.037933	0.0662
99925.000000	109000.0	12593.333333				
582	2017-09-21 06:00:00	9.601667	27.50	0.516583	0.943	0.
→246917						
0.449	0.102425	0.132	0.072600	0.1010	0.036983	0.0761
89841.666667	129000.0	12579.166667				
583	2017-09-21 07:00:00	9.558333	12.80	0.433583	0.758	0.
→221333						
0.384	0.088100	0.176	0.061750	0.1270	0.027700	0.0461
132250.000000	139000.0	26158.333333				
584	2017-09-21 08:00:00	8.098333	10.20	0.531000	0.921	0.
→258667						
0.488	0.119367	0.304	0.083833	0.1990	0.036675	0.0601
105916.666667	113000.0	19191.666667				
585	2017-09-21 09:00:00	5.748333	8.17	0.470917	0.830	0.
→235917						
0.508	0.119925	0.297	0.082808	0.1850	0.038842	0.0761
87166.666667	104000.0	13641.666667				
586	2017-09-21 10:00:00	6.768333	9.85	0.486333	0.980	0.
→254250						
0.409	0.112050	0.233	0.076083	0.1610	0.035208	0.0620
82975.000000	93700.0	13583.333333				
587	2017-09-21 11:00:00	8.500000	11.90	0.524917	0.875	0.
→254417						
0.519	0.103933	0.147	0.076683	0.1350	0.039750	0.0761
86983.333333	92900.0	15216.666667				
588	2017-09-21 12:00:00	14.343333	22.20	0.401167	0.556	0.
→194917						
0.333	0.095167	0.137	0.066008	0.0991	0.036125	0.0705
98750.000000	106000.0	20175.000000				
589	2017-09-21 13:00:00	21.450000	26.30	0.561250	0.954	0.
→294333						
0.543	0.109992	0.136	0.071283	0.0965	0.038075	0.0613
106916.666667	113000.0	23283.333333				
590	2017-09-21 14:00:00	21.400000	31.20	0.353250	0.551	0.
→207333						
0.328	0.111733	0.216	0.078925	0.1350	0.039167	0.0687
133916.666667	143000.0	32541.666667				
591	2017-09-21 15:00:00	28.891667	41.90	0.299667	0.629	0.
→202750						
0.466	0.097675	0.151	0.065008	0.1010	0.035192	0.0687
148666.666667	151000.0	41091.666667				


```

592 2017-09-21 16:00:00 32.458333 44.50 0.302000 0.822 0.
    ↳202333
0.459 0.092517 0.149 0.068467 0.1250 0.035283 0.0686
147166.666667 151000.0 42008.333333
593 2017-09-21 17:00:00 51.766667 77.20 0.285000 0.341 0.
    ↳189667
0.266 0.095725 0.188 0.066508 0.1390 0.032775 0.0761
155750.000000 159000.0 49458.333333
594 2017-09-21 18:00:00 57.600000 82.00 0.256917 0.425 0.
    ↳179167
0.263 0.097633 0.163 0.071875 0.1160 0.039925 0.0811
170000.000000 176000.0 60341.666667
595 2017-09-21 19:00:00 63.166667 93.10 0.372500 0.795 0.
    ↳229833
0.472 0.110642 0.157 0.074300 0.1050 0.040192 0.0687
180250.000000 186000.0 65750.000000
596 2017-09-21 20:00:00 42.102500 78.90 0.397500 0.833 0.
    ↳226833
0.429 0.089325 0.138 0.069983 0.1050 0.039692 0.0737
178750.000000 186000.0 63175.000000
597 2017-09-21 21:00:00 50.783333 71.70 0.489250 0.788 0.
    ↳317583
0.501 0.121358 0.204 0.077467 0.1470 0.033417 0.0524
175833.333333 177000.0 58350.000000
598 2017-09-21 22:00:00 39.000000 48.80 0.350000 0.536 0.
    ↳200250
0.310 0.094358 0.143 0.070633 0.1080 0.039600 0.0798
167750.000000 176000.0 49366.666667
599 2017-09-21 23:00:00 28.825000 42.10 0.465000 0.861 0.
    ↳257833
0.506 0.113575 0.168 0.078108 0.1110 0.045125 0.0733
150083.333333 160000.0 37525.000000
600 2017-09-22 00:00:00 NaN NaN NaN NaN NaN
    ↳NaN
NaN NaN NaN NaN NaN NaN
NaN NaN NaN

```

	E>2.0_max	Speed	Density	Radio Flux 10.7
175	8950.0	NaN	NaN	140.0
176	5850.0	NaN	NaN	140.0
177	4480.0	NaN	NaN	140.0
178	4710.0	NaN	NaN	140.0
179	7430.0	NaN	NaN	140.0
180	7420.0	NaN	NaN	140.0
181	7540.0	NaN	NaN	140.0
182	10500.0	NaN	NaN	140.0
577	41400.0	453.0	1.9	NaN

578	36000.0	453.0	1.9	NaN
579	28600.0	450.0	2.0	NaN
580	21900.0	452.0	2.1	NaN
581	15100.0	445.0	1.7	NaN
582	23800.0	443.0	1.5	NaN
583	28800.0	434.0	1.5	NaN
584	21300.0	431.0	1.7	NaN
585	18500.0	447.0	1.0	NaN
586	16300.0	438.0	1.0	NaN
587	17100.0	419.0	1.7	NaN
588	22500.0	414.0	1.8	NaN
589	24500.0	397.0	1.9	NaN
590	37800.0	382.0	2.1	NaN
591	43400.0	381.0	2.2	NaN
592	43700.0	375.0	2.3	NaN
593	51200.0	365.0	3.0	NaN
594	65900.0	375.0	2.8	NaN
595	66800.0	394.0	1.3	NaN
596	70500.0	379.0	1.8	NaN
597	60300.0	380.0	1.8	NaN
598	54400.0	387.0	1.1	NaN
599	44400.0	380.0	1.0	NaN
600	NaN	381.0	1.0	NaN

Відповідь 4.3: 601 рядок для outer, 577 рядків для inner. Обираю outer, бо для inner деякі дані втрачаються через пропуски в клітинках.

5.0.4 Завдання 4.4 — поділ на групи

Щоб далі було що з чим порівнювати, поділіть години на три групи за значенням **максимуму потоку P>10 за годину**:

Група	Умова
тиха	менше 10
активна	від 10 до 1000
дуже активна	1000 і більше

Додайте до M стовпець група і виведіть, скільки годин потрапило в кожну.

Це не вигадані числа: саме такі пороги використовує метеослужба космічної погоди NOAA. Чому вони саме такі — питання не сьогоднішньої роботи.

```
[28]: M['група'] = pd.cut(M['P>10_max'], bins=[-float('inf'), 10, 1000,
    ↪ float('inf')], labels=['тиха', 'активна', 'дуже
    ↪ активна'], right=False)
print(M['група'].value_counts())
M[M['група'] == 'дуже активна']
```

```

група
тиха          411
активна       171
дуже активна  18
Name: count, dtype: int64

```

```

[28]:          ts      P>1_mean  P>1_max      P>5_mean  P>5_max  □
      ↪ P>10_mean
P>10_max  P>30_mean  P>30_max  P>50_mean  P>50_max  P>100_mean  □
      ↪ P>100_max
E>0.8_mean  E>0.8_max      E>2.0_mean  \
330 2017-09-10 18:00:00  1937.500000  2260.0  951.916667  1150.0  □
      ↪ 834.750000
1010.0  289.833333      356.0  124.916667      153.0  38.325000  □
      ↪ 44.9
166416.666667  171000.0  11941.666667
334 2017-09-10 22:00:00  2770.000000  3500.0  1415.000000  1540.0  □
      ↪ 1019.916667
1070.0  349.083333      377.0  171.250000      191.0  52.933333  □
      ↪ 61.5
148000.000000  167000.0  11500.000000
336 2017-09-11 00:00:00  4891.666667  6400.0  1660.833333  1920.0  □
      ↪ 1102.916667
1230.0  366.166667      398.0  172.333333      185.0  48.850000  □
      ↪ 51.7
144500.000000  152000.0  11150.833333
337 2017-09-11 01:00:00  3895.000000  5350.0  1823.333333  1960.0  □
      ↪ 1167.500000
1230.0  387.083333      399.0  180.833333      185.0  50.041667  □
      ↪ 51.2
104200.000000  129000.0  6556.666667
338 2017-09-11 02:00:00  3224.166667  3480.0  1672.500000  1760.0  □
      ↪ 1086.666667
1110.0  376.916667      384.0  177.750000      180.0  48.100000  □
      ↪ 49.3
98750.000000  116000.0  5792.500000
339 2017-09-11 03:00:00  2970.833333  3110.0  1616.666667  1690.0  □
      ↪ 1080.833333
1120.0  377.333333      388.0  174.666667      178.0  44.275000  □
      ↪ 45.7
101900.000000  109000.0  6407.500000
340 2017-09-11 04:00:00  2975.000000  3540.0  1797.500000  2030.0  □
      ↪ 1071.666667
1100.0  363.666667      372.0  167.583333      171.0  41.425000  □
      ↪ 42.9
77733.333333  92100.0  4571.666667

```

341 2017-09-11 05:00:00 3726.666667 4010.0 2120.833333 2240.0 □
 ↪1144.166667
 1200.0 365.333333 374.0 162.833333 165.0 38.125000 □
 ↪40.2
 52458.333333 60200.0 3320.833333
 342 2017-09-11 06:00:00 3605.833333 3860.0 1993.333333 2120.0 □
 ↪1114.166667
 1150.0 356.916667 363.0 155.166667 158.0 34.741667 □
 ↪35.8
 40491.666667 46800.0 2438.333333
 343 2017-09-11 07:00:00 3469.166667 3600.0 1903.333333 1960.0 □
 ↪1082.500000
 1100.0 345.666667 351.0 146.916667 151.0 31.733333 □
 ↪33.2
 38700.000000 46100.0 2655.833333
 344 2017-09-11 08:00:00 3353.333333 3570.0 1862.500000 2000.0 □
 ↪1080.833333
 1150.0 338.416667 348.0 140.083333 143.0 28.958333 □
 ↪30.7
 37708.333333 52700.0 2734.166667
 345 2017-09-11 09:00:00 4557.500000 5140.0 2306.666667 2480.0 □
 ↪1236.666667
 1270.0 347.083333 355.0 135.833333 139.0 26.675000 □
 ↪27.4
 39583.333333 47600.0 3219.166667
 346 2017-09-11 10:00:00 5018.333333 5210.0 2265.000000 2420.0 □
 ↪1190.833333
 1260.0 331.750000 347.0 128.583333 135.0 24.508333 □
 ↪25.7
 35016.666667 41600.0 2964.166667
 347 2017-09-11 11:00:00 4955.833333 5350.0 2298.333333 2460.0 □
 ↪1243.333333
 1330.0 337.833333 354.0 126.500000 130.0 22.925000 □
 ↪23.4
 26416.666667 28900.0 2613.333333
 348 2017-09-11 12:00:00 4937.500000 5120.0 2240.000000 2400.0 □
 ↪1230.833333
 1330.0 327.750000 352.0 118.916667 127.0 20.650000 □
 ↪21.7
 31441.666667 34300.0 2619.166667
 349 2017-09-11 13:00:00 4525.000000 4860.0 2048.333333 2130.0 □
 ↪1136.666667
 1170.0 303.833333 314.0 109.333333 112.0 18.633333 □
 ↪19.8
 42341.666667 52100.0 3062.500000

```

350 2017-09-11 14:00:00 4675.833333 5320.0 1936.666667 2090.0
↪1086.666667
1130.0 286.000000 295.0 100.883333 104.0 16.475000
↪17.8
61983.333333 70700.0 4255.833333
351 2017-09-11 15:00:00 3988.333333 4270.0 1743.333333 1840.0
↪1024.166667
1060.0 264.166667 275.0 91.400000 95.7 14.533333
↪15.5
79850.000000 89900.0 5774.166667

```

	E>2.0_max	Speed	Density	Radio Flux 10.7	група
330	12400.0	469.0	0.9	100.0	дуже активна
334	14600.0	552.0	2.6	100.0	дуже активна
336	13100.0	566.0	4.8	80.0	дуже активна
337	9310.0	575.0	3.2	80.0	дуже активна
338	7060.0	564.0	4.0	80.0	дуже активна
339	6720.0	557.0	4.6	80.0	дуже активна
340	5870.0	556.0	3.6	80.0	дуже активна
341	3680.0	556.0	4.3	80.0	дуже активна
342	2760.0	558.0	4.5	80.0	дуже активна
343	3160.0	556.0	4.2	80.0	дуже активна
344	3390.0	554.0	3.9	80.0	дуже активна
345	3930.0	555.0	4.3	80.0	дуже активна
346	3260.0	556.0	5.2	80.0	дуже активна
347	2830.0	556.0	5.5	80.0	дуже активна
348	2750.0	555.0	5.0	80.0	дуже активна
349	3820.0	555.0	4.7	80.0	дуже активна
350	5190.0	556.0	3.6	80.0	дуже активна
351	6850.0	557.0	2.7	80.0	дуже активна

Самоперевірка етапу 4.

```

[29]: assert M is not None, 'таблиця M не побудована'
assert 'ts' in M.columns, 'у таблиці M потрібен стовпець ts'
assert 'група' in M.columns, 'у таблиці M потрібен стовпець «група»'
assert 590 <= len(M) <= 605, 'очікується близько 600 рядків, а є %d —
↪перевірте об'єднання' % len(M)
print('Етап 4 пройдено:', len(M), 'рядків')

```

Етап 4 пройдено: 601 рядків

6 Етап 5. П'ять графіків — п'ять питань

Графік будується як **відповідь на питання**, а не «щоб був».

Вимоги до кожного графіка:

- підписані обидві осі, з одиницями вимірювання;
- заголовок;
- під графіком, у текстовій клітинці, — **одне речення про дані**. Не «на графіку зображено потік протонів», а твердження на кшталт «більшість годин потік не перевищує 60, але кілька годин дають значення у сотні разів більші».

Не кожне питання має відповідь «так, зв'язок є». Якщо на графіку нічого не видно — так і напишіть. Відсутність зв'язку — це теж результат, і чесно про неї написати важливіше, ніж підбирати ракурс, у якому «щось наче проглядається».

6.0.1 Графік 1. Як розподілені значення вашого потоку?

Побудуйте гістограму вашого потоку (середнє за годину). Порахуйте окремо середнє й медіану цього стовпця.

Якщо гістограма вийде нечитабельною — стовпчик біля нуля і порожньо далі — не залишайте так. Спробуйте побудувати гістограму від логарифма значень (пр. $\log_{10}$) і подивіться, що зміниться.

Питання: чому середнє й медіана так сильно відрізняються?

```
[30]: print("Середнє:", M['P>30_mean'].mean())
      print("Медіана:", M['P>30_mean'].median())
      P30=M['P>30_mean']
      P30_positive=P30[P30 > 0]
      P30_log=np.log10(P30_positive)

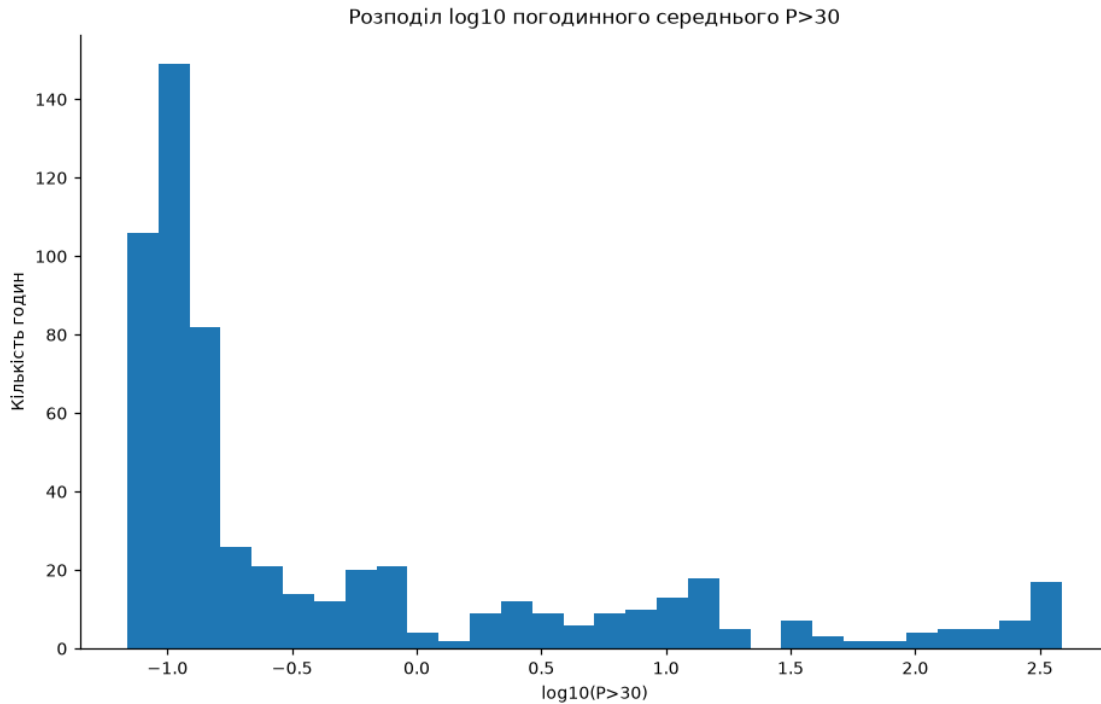
      plt.figure(figsize=(10, 6))
      plt.hist(P30_log, bins=30)

      plt.xlabel('log10(P>30)')
      plt.ylabel('Кількість годин')
      plt.title('Розподіл log10 погодинного середнього P>30')

      plt.show()
```

Середнє: 18.831272835858584

Медіана: 0.137825



Висновок: В основному значення потоку є дуже малими, але через деякі великі значення середнє стає набагато більшим, ніж медіана.

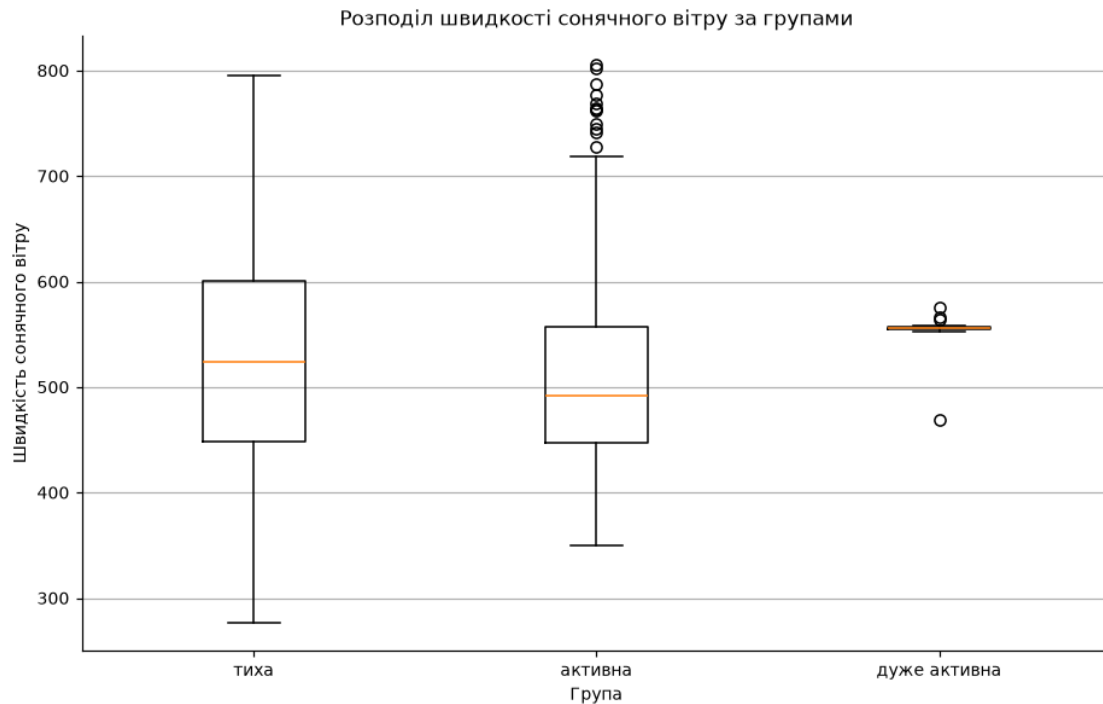
6.0.2 Графік 2. Чи відрізняється швидкість вітру між групами годин?

Побудуйте діаграму розмаху (boxplot) швидкості сонячного вітру окремо для кожної групи з завдання 4.4.

```
[31]: groups = ['тиха', 'активна', 'дуже активна']
data = [
    M.loc[M['група'] == group, 'Speed'].dropna()
    for group in groups
]

plt.figure(figsize=(10, 6))
plt.boxplot(data, tick_labels=groups)

plt.xlabel('Група')
plt.ylabel('Швидкість сонячного вітру')
plt.title('Розподіл швидкості сонячного вітру за групами')
plt.grid(axis='y')
plt.show()
```

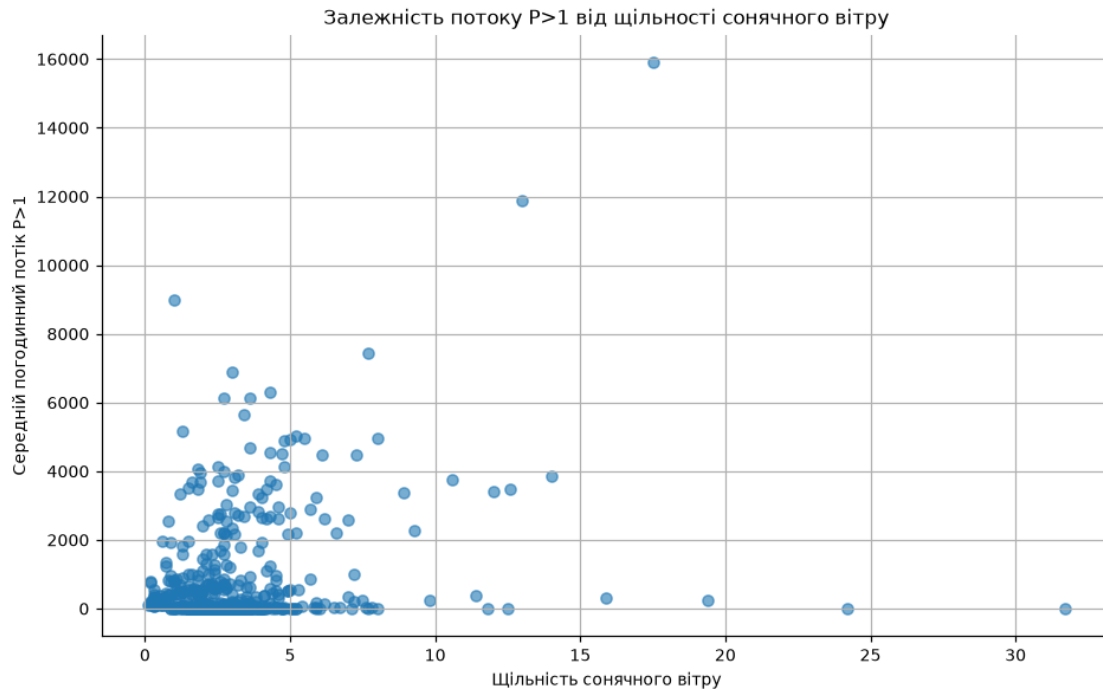


Висновок: Відрізняється, в кожній групі свій діапазон швидкості вітру.

6.0.3 Графік 3. Чи пов'язані змінні вашої пари?

Побудуйте діаграму розсіювання для пари з вашого завдання. Порахуйте коефіцієнт кореляції між ними.

```
[32]: data = M[['Density', 'P>1_mean']].dropna()
plt.figure(figsize=(10, 6))
plt.scatter(data['Density'], data['P>1_mean'], alpha=0.6)
plt.xlabel('Щільність сонячного вітру')
plt.ylabel('Середній погодинний потік P>1')
plt.title('Залежність потоку P>1 від щільності сонячного вітру')
plt.grid(True)
plt.show()
correlation = data['Density'].corr(data['P>1_mean'])
print('Коефіцієнт кореляції:', correlation)
```

Коефіцієнт кореляції: 0.28528892999766575

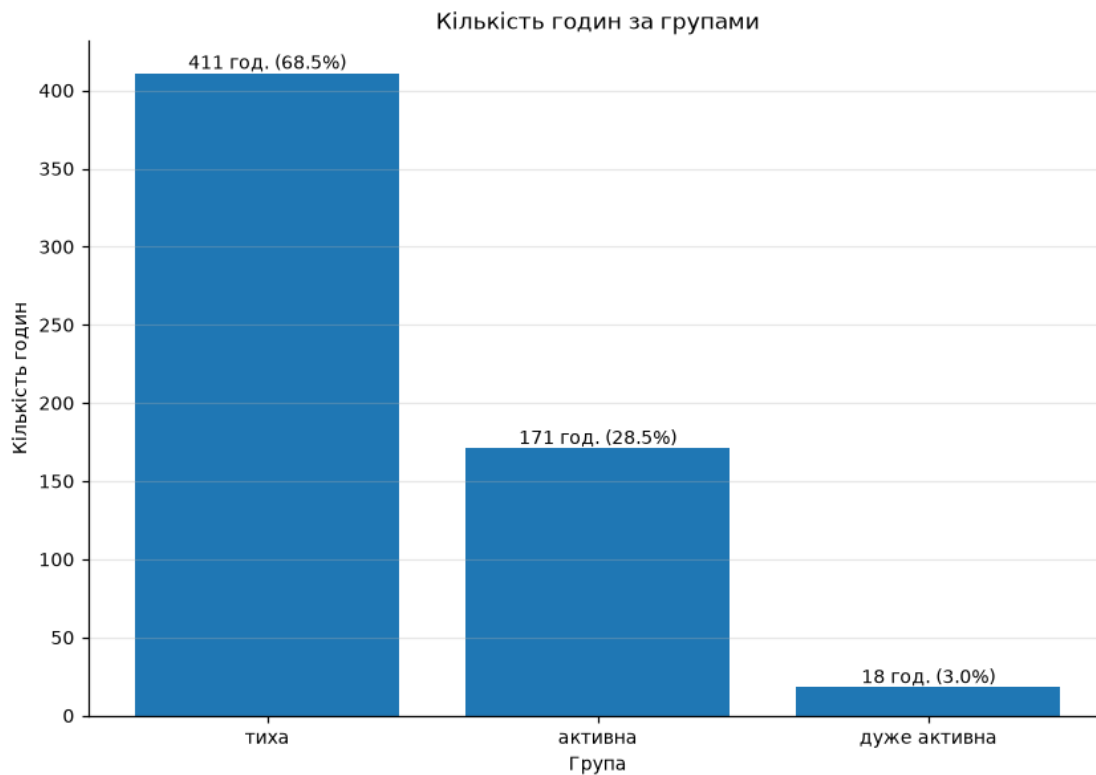
Висновок: Є кореляція в позитивний бік.

6.0.4 Графік 4. Скільки годин у кожній групі?

Побудуйте стовпчикову діаграму кількості годин за групами. Підпишіть кожен стовпчик кількістю годин і відсотком від загальної кількості.

```
[33]: counts = (M['група'].value_counts().reindex(groups, fill_value=0))
total = counts.sum()
percent = counts / total * 100
plt.figure(figsize=(9, 6))
bars = plt.bar(groups, counts)
for bar, count, pct in zip(bars, counts, percent):
    plt.text(
        bar.get_x() + bar.get_width() / 2,
        bar.get_height(),
        f'{count} год. ({pct:.1f}%)',
        ha='center',
        va='bottom'
    )
plt.xlabel('Група')
plt.ylabel('Кількість годин')
plt.title('Кількість годин за групами')
```

```
plt.grid(axis='y', alpha=0.3)
plt.show()
```



Висновок: Набагато більше годин в тихій групі (411). Активна група має 171 годину, а дуже активна - дуже мала група (тільки 18 годин)

6.0.5 Графік 5. Як усе це виглядало в часі?

Побудуйте лінійний графік максимуму потоку $P > 10$ за годину за весь період спостережень.

Якщо крива виявиться нечитабельною — згадайте, що допомогло в графіку 1.

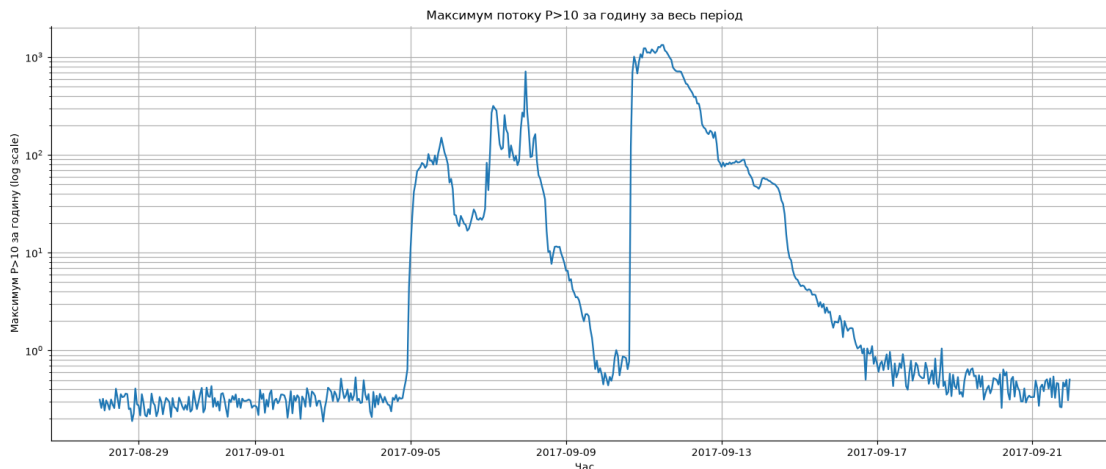
```
[34]: data = M[['ts', 'P>10_max']].dropna()
data = data[data['P>10_max'] > 0]

plt.figure(figsize=(14, 6))
plt.plot(data['ts'], data['P>10_max'])

plt.yscale('log')

plt.xlabel('Час')
plt.ylabel('Максимум P>10 за годину (log scale)')
```

```
plt.title('Максимум потоку P>10 за годину за весь період')
plt.grid(True, which='both')
plt.tight_layout()
plt.show()
```



Висновок: Спостерігалась висока сонячна активність від 5 вересня до 9 вересня та від 10 вересня до 17 вересня 2017 року. Поза цими датами сонячна активність залишалась низькою.

7 Етап 6. Два графіки одних і тих самих даних

Нижче побудовано два графіки. Дані в них **одні й ті самі** — це той самий стовпець з тієї самої таблиці, жодне число не змінено.

Виконайте клітинку й подивіться на обидва.

```
[35]: СТОВПЕЦЬ = 'P>10_max' # <- впишіть назву свого стовпця з МАКСИМУМОМ
      ↪ P>10 за годину

вікно = M[(M['ts'] >= '2017-08-28') & (M['ts'] < '2017-09-05')]

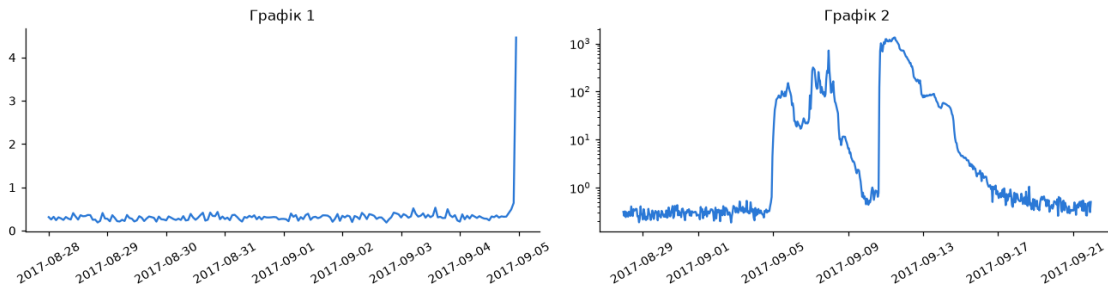
fig, ax = plt.subplots(1, 2, figsize=(12, 3.2))

ax[0].plot(вікно['ts'], вікно[СТОВПЕЦЬ], color='#2a78d6')
ax[0].set_title('Графік 1')
ax[0].tick_params(axis='x', rotation=30)

ax[1].plot(M['ts'], M[СТОВПЕЦЬ], color='#2a78d6')
ax[1].set_yscale('log')
```

```
ax[1].set_title('Графік 2')
ax[1].tick_params(axis='x', rotation=30)

fig.tight_layout()
plt.show()
```



7.0.1 Завдання 6.1

Дайте відповідь на три питання:

1. Який висновок про космічну погоду у вересні 2017 року зробить людина, яка бачила **тільки перший** графік?
2. Який висновок зробить людина, яка бачила **тільки другий**?
3. Чим ці два графіки відрізняються між собою? Назвіть **дві** відмінності — вони обидві є в коді вище.

Жодне число в обох графіках не підроблене. Різниця виникла тільки через те, як їх побудували.

Відповідь 6.1:

1. Висновок з першого графіка: активність набагато збільшилась (в 7-8 разів) від 5 вересня 2017 року
2. Висновок з другого графіка: активність була досить низькою, крім 5-9 та 10-14 вересня 2017 року
3. Дві відмінності між ними: діапазон дат, масштаб показу даних.

7.0.2 Завдання 6.2

Уявіть, що вам потрібно **одним графіком** чесно показати, що відбувалося в цьому періоді. Побудуйте його самі й підпишіть заголовком, який формулює висновок.

Поясніть у клітинці нижче, чому ви обрали саме такий вигляд.

```
[36]: вікно = M[
        (M['ts'] >= '2017-09-03') &
        (M['ts'] < '2017-09-18')
    ].copy()
```

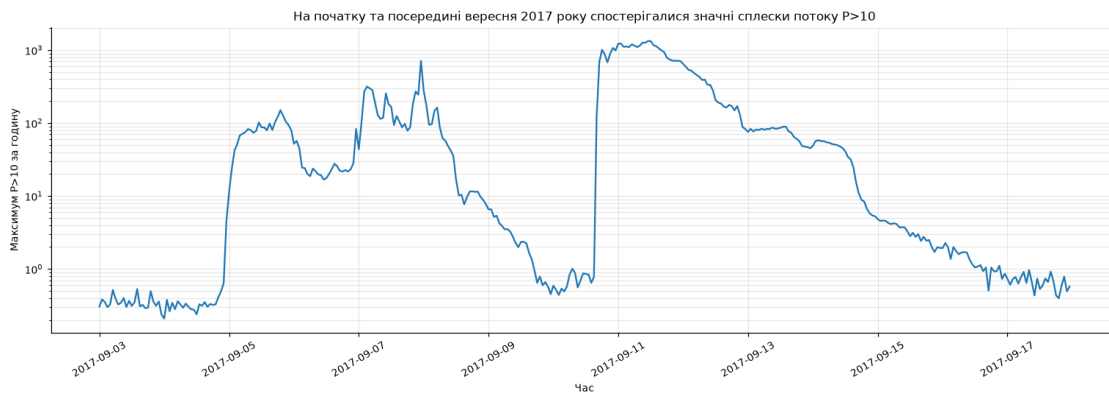
```

вікно = вікно[вікно['P>10_max'] > 0]
plt.figure(figsize=(14, 5))
plt.plot(
    вікно['ts'],
    вікно['P>10_max']
)
plt.yscale('log')

plt.xlabel('Час')
plt.ylabel('Максимум P>10 за годину')
plt.title('На початку та посередині вересня 2017 року спостерігалися
↪значні сплески потоку P>10')

plt.xticks(rotation=30)
plt.grid(True, which='both', alpha=0.3)
plt.tight_layout()
plt.show()

```



Відповідь 6.2: Логарифмізація значень допомагає краще показати коливання значень протягом певних періодів часу (значення стискаються). Протягом саме цього діапазону часу відбуваються сплески, і можна показати де вони почалися та закінчилися.

8 Етап 7. Три спостереження та ваше питання

8.0.1 Завдання 7.1

Запишіть **три спостереження** про ці дані, які ви зробили під час роботи.

Спостереження — це твердження, яке спирається на конкретне число або графік з вашого ноутбука. Не «дані цікаві», а, наприклад, «максимальне значення потоку у 2000 разів більше за медіанне, і всі такі години припадають на одні й ті самі три доби».

Для кожного вкажіть, **де саме** у вашій роботі його видно.

Спостереження 1: Було тільки 18 годин, протягом яких спостерігався дуже активний потік $P > 10$ ($P > 10_max > 1000$), причому всі з них спостерігались в межах однієї доби (2017-09-10 18:00:00 - 2017-09-11 18:00:00). Код - завдання 4.4

Спостереження 2: В активній групі дуже багато нетипових значень швидкості вітру (700-800). Код - завдання 5, графік 2

Спостереження 3: Найменші значення потоку в $P > 100$, причому набагато менші, ніж в інших потоках. Код - завдання 3.3

8.0.2 Завдання 7.2 — ваше індивідуальне питання

Дайте відповідь на додаткове питання зі свого завдання. Відповідь має спиратися на обчислення: наведіть код і результат, а потім сформулюйте висновок словами.

```
[37]: daily_P30 = (
        M.assign(date=M['ts'].dt.date)
        .groupby('date')['P>30_mean']
        .mean()
    )

    print("Найбільше:", daily_P30.idxmax(), daily_P30.max())
    print("Найменше:", daily_P30.idxmin(), daily_P30.min())
```

Найбільше: 2017-09-11 292.93055555555554

Найменше: 2017-09-01 0.08973368055555557

Відповідь 7.2: 11 вересня 2017 року спостерігалось найбільше середнє значення потоку $P > 30$, 1 вересня - найменше.

9 Етап 8. Декларація про використання інструментів штучного інтелекту

Користуватися мовними моделями під час виконання цієї роботи **дозволено**. Це нормальна інженерна практика, і курс про штучний інтелект не має підстав її забороняти.

Обов'язковою є прозорість: ви вказуєте, де саме і як скористалися інструментом — так само, як у науковій статті вказують використане програмне забезпечення.

Задекларована допомога порушенням не є.

Порушенням є інше: здати роботу, яку ви не можете пояснити. На захисті вас попросять показати конкретний рядок вашого коду, пояснити, чому він саме такий, і внести в нього невелику зміну на місці. Якщо код згенеровано, але ви його розумієте — робота зарахована. Якщо не розумієте — не зарахована, незалежно від того, хто його написав.

9.0.1 Завдання 8.1

Заповніть словник. Для кожного етапу оберіть одне з формулювань:

- 'ні' — інструментами ШІ не користувався;
- 'синтаксис' — питав про назву функції або синтаксис, код писав сам;
- 'код, перевірено' — код згенеровано, я його прочитав, перевірів і розумію;
- 'текст, перероблено' — згенеровано чернетку тексту, яку я переписав по суті;
- 'текст, як є' — згенерований текст залишено майже без змін.

Останній варіант не заборонений, але прямо впливає на оцінку: бали ставляться за висновки, а не за наявність абзацу.

```
[38]: ІНСТРУМЕНТИ_ШІ = 'ChatGPT'      # напр. 'ChatGPT' або 'не□
      ↪використовував'

ДЕКЛАРАЦІЯ = {
    'етапи 1-2, читання файлу та розбір таблиць': 'синтаксис',
    'етап 3, паспорт даних і дивні значення':      'код, перевірено',
    'етап 4, об'єднання таблиць':                  'код, перевірено',
    'етап 5, побудова графіків':                    'код, перевірено',
    'етап 5, формулювання висновків':               'ні',
    'етапи 6-7, порівняння графіків і спостереження': 'код,□
    ↪перевірено',
}

# Що саме ви перевірили за згенерованим кодом чи текстом — конкретно:
# які числа звірили, що переписали, де знайшли помилку.
ЩО_ПЕРЕВІРЕНО = 'Знайшов помилку для підпису колонок під графіком□
    ↪коробок (бот написав "labels" замість "tick_labels"). Також□
    ↪перевірів будову графіків.'

# True означає: я можу пояснити кожен рядок свого коду, відтворити□
    ↪будь-який
# результат цієї роботи та внести зміни в код на вимогу під час□
    ↪захисту.
ПІДТВЕРДЖУЮ = True
```

```
[39]: # --- перевірка повноти декларації: клітинку не редагувати□
      ↪-----
ДОЗВОЛЕНІ = {'ні', 'синтаксис', 'код, перевірено', 'текст,□
    ↪перероблено', 'текст, як є'}

assert ІНСТРУМЕНТИ_ШІ.strip(), 'Вкажіть інструменти або напишіть «не□
    ↪використовував»'
порожні = [k for k, v in ДЕКЛАРАЦІЯ.items() if not str(v).strip()]
assert not порожні, 'Не заповнено пункти декларації: %s' % '; '.
    ↪join(порожні)
```

```

чужі = {v for v in ДЕКЛАРАЦІЯ.values() if v not in ДОЗВОЛЕНІ}
assert not чужі, 'Недопустимі значення: %s. Оберіть з переліку вище.'
↳ % ', '.join(sorted(чужі))
assert len(ЩО_ПЕРЕВІРЕНО.strip()) >= 120, ('Опишіть конкретніше, що
↳ саме ви перевірили '
                                     '(зараз %d символів,
↳ потрібно щонайменше 120)'
                                     % len(ЩО_ПЕРЕВІРЕНО.
↳ strip()))
assert ПІДТВЕРДЖУЮ is True, 'Підтвердьте, що можете пояснити власний
↳ код на захисті'

print('ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ')
print('=' * 78)
print('%-46s %s' % ('автор:', ПІБ))
print('%-46s %s' % ('інструменти:', ІНСТРУМЕНТИ_ШІ))
print('-' * 78)
for k, v in ДЕКЛАРАЦІЯ.items():
    print('%-46s %s' % (k + ':', v))
print('-' * 78)
print('перевірено автором:')
print(ЩО_ПЕРЕВІРЕНО)
print('=' * 78)
print('Автор підтверджує, що може пояснити кожен рядок коду під час
↳ захисту.')

```

ДЕКЛАРАЦІЯ ПРО ВИКОРИСТАННЯ ІНСТРУМЕНТІВ ШІ

```

=====
автор:                                Колбасюк Данило Іванович
інструменти:                          ChatGPT
-----

```

```

етапи 1-2, читання файлу та розбір таблиць:    синтаксис
етап 3, паспорт даних і дивні значення:        код, перевірено
етап 4, об'єднання таблиць:                    код, перевірено
етап 5, побудова графіків:                     код, перевірено
етап 5, формулювання висновків:                ні
етапи 6-7, порівняння графіків і спостереження: код, перевірено
-----

```

```

перевірено автором:
Знайшов помилку для підпису колонок під графіком коробок (бот написав
↳ "labels"
замість "tick_labels"). Також перевірів будову графіків.
=====

```

Автор підтверджує, що може пояснити кожен рядок коду під час захисту.

10 Крок 9. Збереження результату та фінальна самоперевірка

Об'єднана таблиця знадобиться вам у лабораторних роботах № 3 і № 4 — збережіть її й не втрачайте до кінця семестру.

```
[40]: M.to_csv('master_hourly.csv', index=False)
      print('збережено:', M.shape)
```

збережено: (601, 21)

```
[41]: # --- фінальна самоперевірка
      ↪-----
      перевірки = {
          'ПІБ і варіант заповнено':          bool(ПІБ) and bool(ВАРІАНТ),
          'таблиця А (7200 рядків)':          len(A) == 7200,
          'таблиця В (25 рядків)':            len(B) == 25,
          'таблиця С (601 рядок)':            len(C) == 601,
          'таблиці об'єднано':                M is not None and 590 <= len(M)
          ↪<= 605,
          'стовпець «група» є':              'група' in M.columns,
          'усі дати у 2017 році':            int(pd.to_datetime(M['ts']).dt.
          ↪year.min()) == 2017,
          'декларацію заповнено':            bool(ІНСТРУМЕНТИ_ШІ.strip())
          ↪and ПІДТВЕРДЖУЮ is True,
      }
      for k, v in перевірки.items():
          print(('OK' if v else 'НЕ OK') + ' ' + k)

      if all(перевірки.values()):
          print('\nТехнічна частина роботи виконана.')
          print('Тепер переконайтеся, що заповнені ВСІ клітинки з
          ↪відповідями та висновками,')
          print('і виконайте Kernel → Restart & Run All перед здачею.')
```

OK ПІБ і варіант заповнено
OK таблиця А (7200 рядків)
OK таблиця В (25 рядків)
OK таблиця С (601 рядок)
OK таблиці об'єднано
OK стовпець «група» є
OK усі дати у 2017 році
OK декларацію заповнено

Технічна частина роботи виконана.

Тепер переконайтеся, що заповнені ВСІ клітинки з відповідями та
↪висновками,

і виконайте Kernel → Restart & Run All перед здачею.